EXPERIMENT-2(Skill)

Name: Thanam Anthony Nissy

Rollno: 2520030064

Section: 8

Experiment No. 2(Q2)

Title: Implementation of a Simple Linux Shell using

Aim: To develop a simple Linux shell that continuously accepts user commands, displays a prompt, executes Linux commands, and exits when the user enters the exit command.

Objective:

- To understand the working of a Linux shell.
- To create an interactive command loop.
- To read user input from the keyboard.
- To execute Linux commands.
- To handle exit conditions.
- To understand shell control flow.

Theory:

A shell is a command-line interpreter that acts as an interface between the user and the Linux operating system. It continuously waits for user commands, interprets them, and executes the requested operations.

A shell performs the following functions:

- Displays a command prompt.
- Reads user input.
- Processes the entered command.
- Executes Linux commands.
- Repeats until the user exits.

In this experiment, a simple shell is implemented using the C programming language. The shell repeatedly accepts commands from the user and executes them using the **system()** function. The loop continues until the user enters **exit**.

Algorithm:

1. Start the program.
2. Display the shell prompt.
3. Read a command from the user.
4. Remove the newline character.
5. If the command is empty, display the prompt again.
6. If the command is exit, terminate the program.
7. Otherwise execute the command.
8. Repeat Steps 2–7.
9. Stop.

Functions Used:

Function	Purpose
printf()	Display shell prompt
fgets()	Read user input
strcmp()	Compare strings
system()	Execute Linux command
strcspn()	Remove newline character

Sample Input: pwd

Sample Output:

myshell> pwd

/home/student/Question2

myshell> ls

shell

shell.c

Makefile

README.md

myshell> date

Sat Aug 1 10:35:12 IST 2026

myshell> exit

Exiting Shell...

Observation:

1. The shell continuously displayed the command prompt.
2. User commands were successfully accepted.
3. Linux commands were executed correctly.
4. The shell accepted multi-character commands.
5. The program terminated only after entering exit.
6. The interactive loop worked successfully.

Result:

The simple Linux shell was successfully implemented using C. The program continuously accepted commands, executed Linux commands, displayed the output, and terminated successfully when the user entered the **exit** command.

Part 2: Linux Commands Observation

Aim: To understand the basic Linux commands used while working with a simple Linux shell and to observe their outputs.

Commands Used:

1. uname:

Purpose: Displays information about the Linux operating system.

Observation: Shows the kernel name and version of the operating system.

2. lscpu:

Purpose: Displays CPU architecture and processor details.

Observation: Provides information about CPU model, cores, threads, cache size, and architecture.

3. lsblk:

Purpose: Lists storage devices connected to the system.

Observation: Displays hard disks, SSDs, partitions, and mounted storage devices.

4. ps:

Purpose: Displays currently running processes.

Observation: Shows Process IDs (PIDs), terminal information, execution time, and running commands.

5. top:

Purpose: Monitors system performance in real time.

Observation: Displays CPU usage, memory usage, running processes, and system load dynamically.

Hardware and OS Relationship:

CPU:
The operating system receives commands entered through the shell, creates processes to execute them, schedules CPU time for these processes, and manages their execution efficiently.

Memory:
The operating system allocates memory for the shell program and for every command executed. It also releases the allocated memory after the process completes.

Storage:
The operating system accesses files and directories stored on disk. Commands such as ls, pwd, and cat interact with the file system to retrieve or display information.

Input/Output Devices:
The operating system accepts keyboard input entered by the user in the shell and displays the command output on the monitor through the terminal.

Conclusion: The experiment successfully demonstrated the implementation of a simple Linux shell. The shell continuously accepted user commands, displayed a prompt, executed Linux commands using the system() function, and terminated correctly when the user entered the exit command. The experiment also helped in understanding the basic workflow of a command-line interpreter.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>


#define MAX_INPUT 1024

int main(){

  char input[MAX_INPUT];

  while (1)  {

    printf("myshell> ");
```

```
        if (fgets(input, MAX_INPUT, stdin) == NULL)   {
            break;
        }
        // Remove newline character
        input[strcspn(input, "\n")] = '\0';
        // Ignore empty input
        if (strlen(input) == 0)   {
            continue;
        }
        // Exit command
        if (strcmp(input, "exit") == 0)  {
            printf("Exiting Shell...\n");
            break;
        }
     // Execute Linux command
        system(input);
    }
    return 0;
}
```

OUPUT:

```
teja@Hasini:~$ mkdir Experiment-2
teja@Hasini:~$ cd Experiment-2
teja@Hasini:~/Experiment-2$ nano shell.c
teja@Hasini:~/Experiment-2$ gcc shell.c -o shell
teja@Hasini:~/Experiment-2$ ./shell
myshell> pwd
/home/teja/Experiment-2
myshell> ls
shell  shell.c
myshell> whoami
teja
myshell> date
Sat Aug  1 10:02:38 UTC 2026
myshell> exit
Exiting Shell...
teja@Hasini:~/Experiment-2$
```